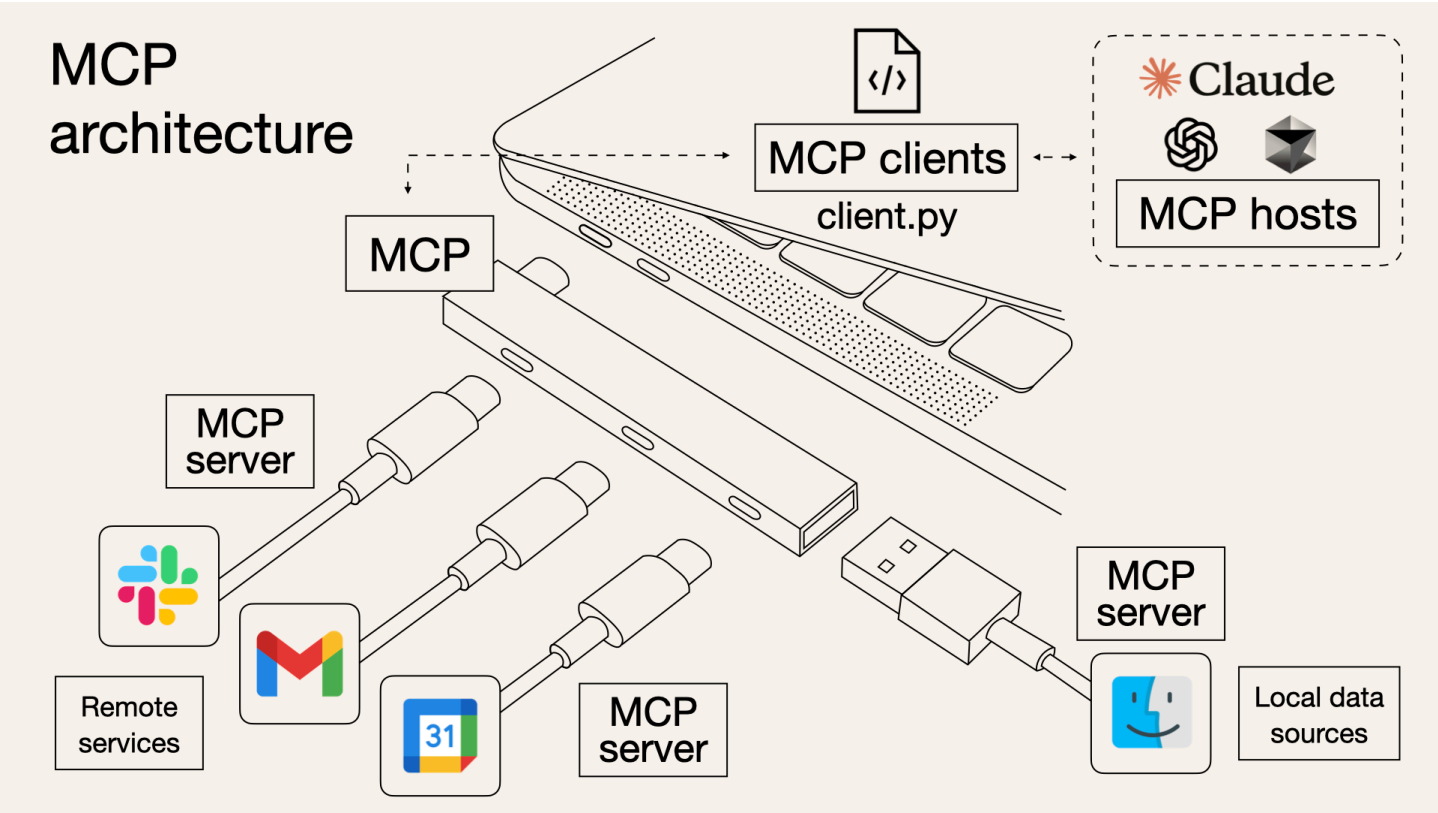


MCP协议

MCP（Model Context Protocol，模型上下文协议）定义了应用程序和 AI 模型之间交换上下文信息的方式。这使得开发者能够以一致的方式将各种数据源、工具和功能连接到 AI 模型（一个中间协议层），就像 USB-C 让不同设备能够通过相同的接口连接一样。MCP 的目标是创建一个通用标准，使 AI 应用程序的开发和集成变得更加简单和统一。



MCP

MCP 就是以更标准的方式让 LLM Chat 使用不同工具，更简单的可视化如下图所示，这样你应该更容易理解“中间协议层”的概念了。Anthropic 旨在实现 LLM Tool Call 的标准。

为什么需要MCP

“ MCP 的出现是 prompt engineering 发展的产物。更结构化的上下文信息对模型的 performance 提升是显著的。我们在构造 prompt 时，希望能提供一些更 specific 的信息（比如本地文件，数据库，一些网络实时信息等）给模型，这样模型更容易理解真实场景中的问题。想象一下没有 MCP 之前我们会怎么做？我们可能会人工从数据库中筛选或者使用工具检索可能需要的信息，手动的粘贴到 prompt 中。随着我们要解决的问题越来越复杂，手工把信息引入到 prompt 中会变得越来越困难。

为了克服手工 prompt 的局限性，许多 LLM 平台（如 OpenAI、Google）引入了 function call 功能。这一机制允许模型在需要时调用预定义的函数来获取数据或执行操作，显著提升了自动化水平。

不同 LLM 平台的 function call API 实现差异较大。所以为了规范Function call的上下文协议，Anthropic 提出了MCP协议。

MCP servers 可以提供三种主要类型的功能：

Resources（资源）：类似文件的数据，可以被客户端读取（如 API 响应或文件内容） **Tools（工具）**：可以被 LLM 调用的函数（需要用户批准） **Prompts（提示）**：预先编写的模板，帮助用户完成特定任务

除了function call， 其他两种类型目前适配的AI厂商几乎没有，所以我们这里也不作展开讨论，主要是关注MCP function call 协议。

MCP上下文协议的实现： chat聊天中多轮对话的过程

第1轮对话

json

```
{
  "model": "gpt-4o-mini",
  "stream": true,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "temperature": 1,
  "top_p": 1,
  "messages": [
    {
      "content": "你是一个计算机领域专家，负责回答用户关于LLM的问题",
      "role": "system"
    },
    {
      "content": "你是谁",
      "role": "user"
    }
  ]
}
```

第2轮对话

json

```
{
  "model": "gpt-4o-mini",
  "stream": true,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "temperature": 1,
  "top_p": 1,
  "messages": [
    {
      "content": "你是一个计算机领域专家，负责回答用户关于LLM的问题",
      "role": "system"
    },
    {
      "content": "你是谁",
      "role": "user"
    },
    {
      "content": "我是一个人工智能助手，专门用来回答关于大型语言模型（LLM）和其他计算机技术的问题。如果你对这些主题有任何",
      "role": "assistant"
    }
  ],
}
```

LLM是无记忆的(无状态的)。通过client端保存回话历史的方式保存状态

dify tool 的聊天过程



发送消息到飞书群

C



可以提供您要发送到飞书群的消息内容吗？

在清明假期给大家送上祝福~

C



1' 已使用 lark_card >

我已经将清明假期的祝福消息发送到飞书群了！希望大家都能拥有一个愉快且安全的假期。

```

[
  {
    "role": "system",
    "text": "weebhook地址: https://open.feishu.cn/open-apis/bot/v2/hook/4fe8ae27-bc83-45ca-b375-d2b5c410750a\n",
    "files": []
  },
  {
    "role": "user",
    "text": "发送消息到飞书群",
    "files": []
  },
  {
    "role": "assistant",
    "text": "可以提供您要发送到飞书群的消息内容吗? ",
    "files": []
  },
  {
    "role": "user",
    "text": "在清明假期给大家送上祝福~",
    "files": []
  },
  {
    "role": "assistant",
    "text": "",
    "files": [],
    "tool_calls": [
      {
        "id": "lark_card",
        "type": "function",
        "function": {
          "name": "lark_card",
          "arguments": "{\"card_content\": \"清明假期快乐! 祝愿大家在假期中放松心情, 享受美好时光. 注意安全, 保持健康!\\n\\n@everyone\", \"title\": \"清明假期祝福\", \"webhook\": \"https://open.feishu.cn/open-apis/bot/v2/hook/4fe8ae27-bc83-45ca-b375-d2b5c410750a\"}"
        }
      }
    ]
  },
  {
    "role": "tool",
    "text": "Card message delivered",
    "files": [],
    "tool_calls": [
      {
        "id": "lark_card",
        "type": "function",
        "function": {
          "name": "lark_card",
          "arguments": "{\"card_content\": \"清明假期快乐! 祝愿大家在假期中放松心情, 享受美好时光. 注意安全, 保持健康!\\n\\n@everyone\", \"title\": \"清明假期祝福\", \"webhook\": \"https://open.feishu.cn/open-apis/bot/v2/hook/4fe8ae27-bc83-45ca-b375-d2b5c410750a\"}"
        }
      }
    ]
  }
]

```

dify-agent-message

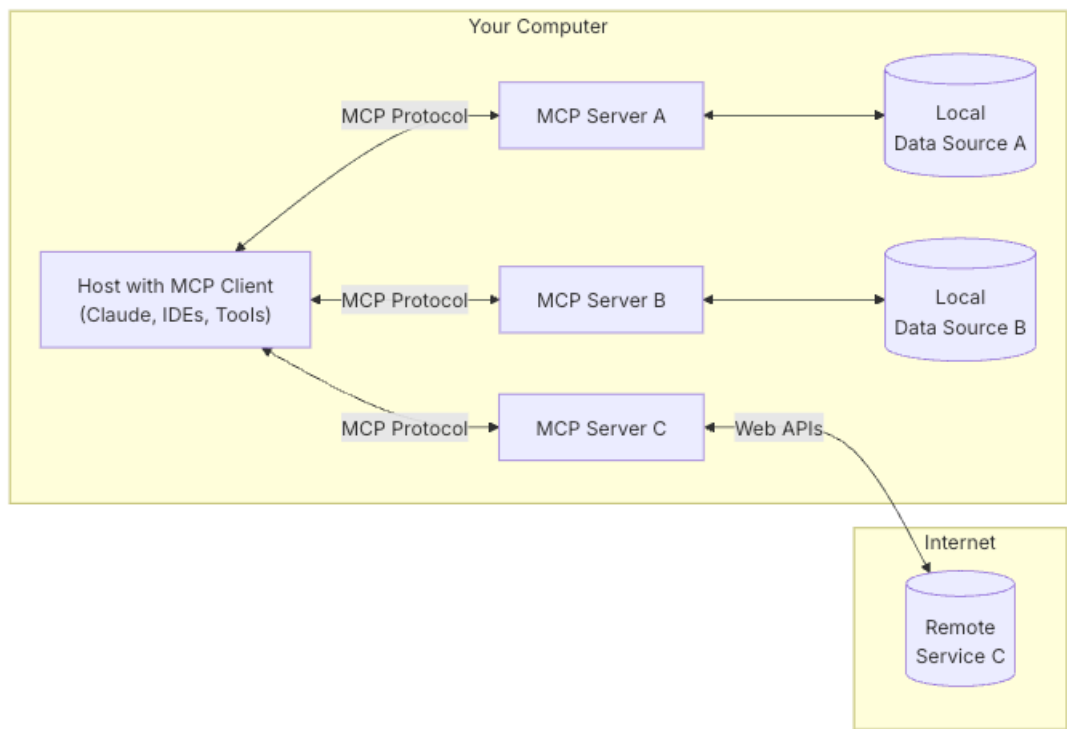
[note]dify实现方式未必是按照标准的MCP实现, 但其实也可以实现相似的功能

MCP规范

MCP tool做了什么规范

- 1. MCP 规范明确了角色定义
- 2. MCP tool规范了工具的context结构
- 3. MCP tool规范了上下文协议过程

1. mcp角色



MCP角色

- **Host:** Claude Desktop 作为 Host，负责接收你的提问并与 Claude 模型交互。
- **Client:** 当 Claude 模型决定需要访问你的文件系统时，Host 中内置的 MCP Client 会被激活。这个 Client 负责与适当的 MCP Server 建立连接。
- **Server:** 在这个例子中，文件系统 MCP Server 会被调用。它负责执行实际的文件扫描操作，访问你的桌面目录，并返回找到的文档列表。

2. MCP tool规范了工具的context结构

server

python

```
# server.py
from mcp.server.fastmcp import FastMCP

from mcp_server.sse.gaode_wether import gaode_weather_async

# Create an MCP server
mcp = FastMCP(
    "Demo",
    host="0.0.0.0",
```

```

    port=9876,
    log_level="INFO",
)

# resource
@mcp.resource("config://app")
def get_config() -> str:
    """Static configuration data"""
    return "Ann configuration here"

```

2. client

python

```

# client.py
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
OPENAI_BASE_URL = os.getenv("OPENAI_BASE_URL")
OPENAI_MODEL=os.getenv("OPENAI_MODEL")

class MCPClient:
    def __init__(self):
        # Initialize session and client objects
        self.session: Optional[ClientSession] = None
        self.exit_stack = AsyncExitStack()
        self.openai = AsyncOpenAI(api_key=os.getenv("OPENAI_API_KEY"), base_url=os.getenv("OPENAI_BASE_U

    async def connect_to_sse_server(self, server_url: str):
        """Connect to an MCP server running with SSE transport"""
        # Store the context managers so they stay alive
        self._streams_context = sse_client(url=server_url)
        streams = await self._streams_context.__aenter__()

        self._session_context = ClientSession(*streams)

```

客户端实现mcp协议

python

```

    async def process_query(self, query: str) -> str:
        """Process a query using OpenAI API and available tools"""
        messages = [
            {
                "role": "user",
                "content": query
            }
        ]

        response = await self.session.list_tools()
        available_tools = [{
            "type": "function",
            "function": {
                "name": tool.name,

```

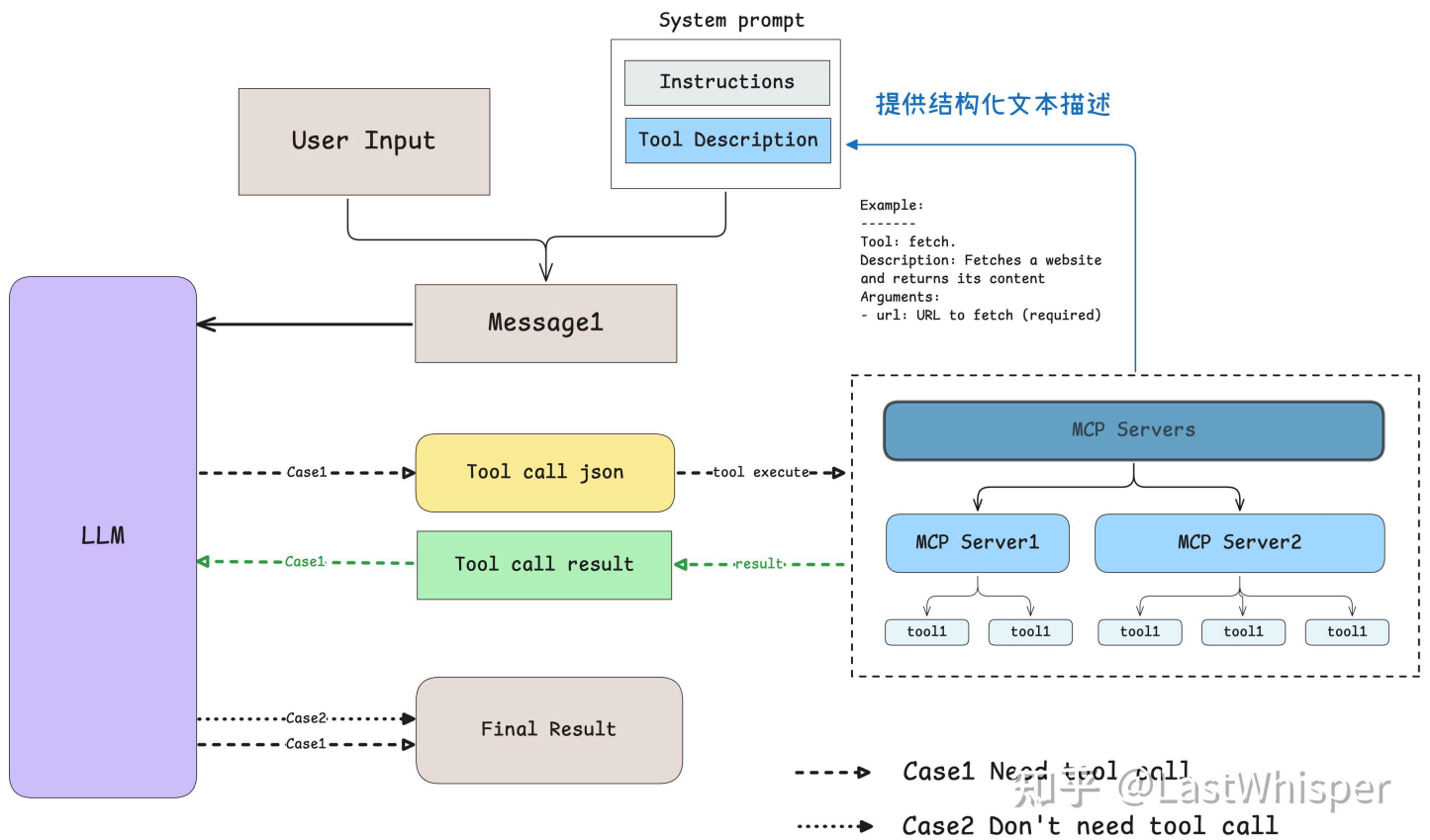
```

        "description": tool.description,
        "parameters": tool.inputSchema
    }
} for tool in response.tools]

# Initial GPT API call

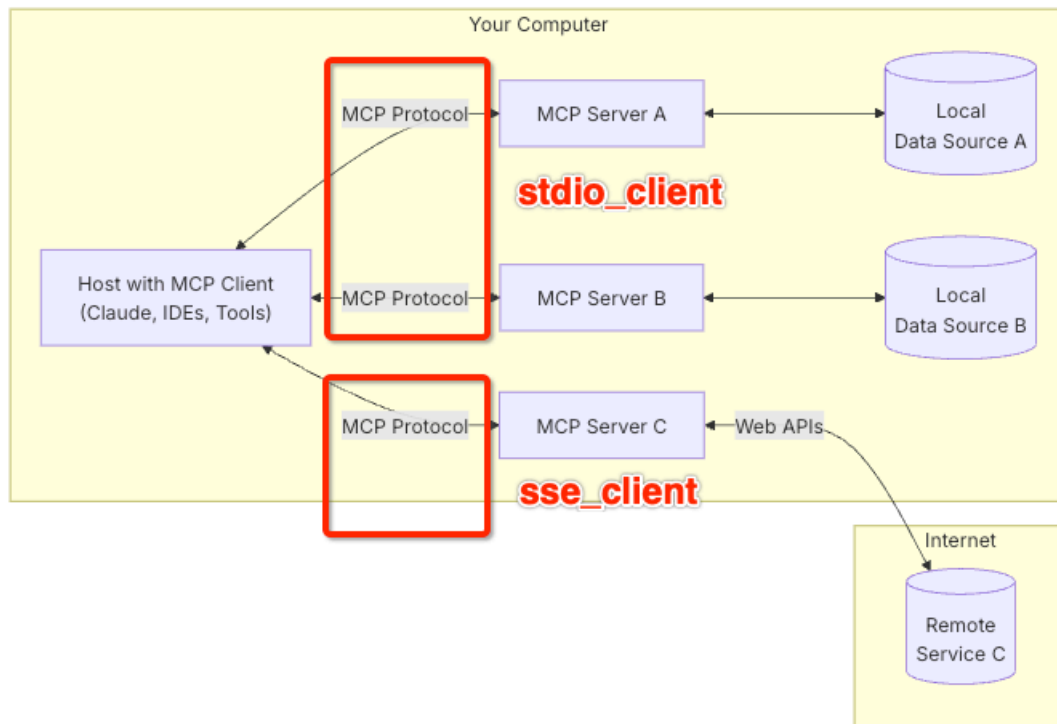
```

3. mcp协议过程



mcp协议过程

mcp client的两种通信方式



mcp_client

实现一个mcp server,并使用dify访问它

[dify-高德天气](#)

参考地址

1. [MCP官网](#)
2. [awesome-mcp-servers](#)
3. [mcpservers](#)

版权所有

本文链接: <http://localhost:8082/article/y1h2iyng/>

许可证: [署名-非商业性-禁止演绎 4.0 国际 \(CC-BY-NC-ND-4.0\)](#)